

1 Piscina

In questo esercizio, vi chiediamo di usare la classe `Semaphore` di JAVA che propone un'implementazione dei semafori. Potete consultare online la documentazione di questa classe e in particolare le tre funzioni seguenti che rappresentano l'uso dei semafori visto a lezione:

- il costruttore `Semaphore(int permits)` per creare un nuovo semaforo inizializzato a `permits`;
- la funzione `void acquire()` che corrisponde al `wait` visto a lezione;
- la funzione `void release()` che corrisponde al `signal` visto a lezione.

Consideriamo una piscina pubblica con N_S spogliatoi individuali e N_C armadietti per lasciare i propri vestiti. Un cliente che si presenta alla piscina segue le seguenti fasi:

- Prende la chiave di uno spogliatoio;
- Prende la chiave di un armadietto;
- Si cambia nello spogliatoio;
- Libera lo spogliatoio;
- Mette i suoi vestiti nell'armadietto;
- Ridà la chiave dello spogliatoio;
- Nuota (tenendosi la chiave dell'armadietto);
- Prende la chiave di uno spogliatoio;
- Recupera i suoi vestiti nel armadietto;
- Si riveste nello spogliatoio;
- Libera lo spogliatoio;
- Rida le chiave dello spogliatoio e dell'armadietto.

Lo scopo dell'esercizio è proporre un'implementazione in JAVA di un programma che simula il comportamento dei clienti in questa piscina.

- Creare un programma JAVA che simuli tanti clienti concorrenti. La disponibilità degli spogliatoi e degli armadietti sarà rappresentata da due semafori condivisi tra i diversi thread che rappresentano i clienti.
- Testare il vostro programma con diversi numeri di clienti, di spogliatoi e di armadietti. In particolare, provate a trovare casi in cui si verifica un deadlock (i.e. tutti i clienti sono bloccati).
- Proponete una modifica minore nel comportamento dei clienti per evitare questo deadlock.

2 Cioccolatini

Consideriamo una scatola che può contenere al massimo $P > 0$ cioccolatini. C'è un pasticciere che in ciclo riempie la scatola con dei cioccolatini, ma la riempie direttamente con P cioccolatini e quindi lo può fare solo quando la scatola è vuota. Ci sono anche dei mangiatori di cioccolatini che in ciclo prendono un cioccolatino dalla scatola e lo mangiano e se la scatola è vuota, aspettano che ci siano di nuovo dei cioccolatini.

Proponete in JAVA una modellazione di questo sistema, considerando che vogliamo un processo per il pasticciere e un processo per i mangiatori e che prenderemo tre mangiatori e P uguale a 5.

3 Consegna

Per la consegna, creare un file `zip` con tutti vostri file. Lo `zip` dovrà anche contenere un file denominato `partecipanti.txt` dove gli nomi di chi ha partecipato alla consegna (questo anche se siete da solo a farla).